

第十章：目标代码生成

概述

目标语言

四元式到目标指令的翻译

寄存器分配

临时变量地址分配

目标代码生成概述

◆ 目标代码生成的任务

- 给变量分配实际地址，寄存器分配，生成管理活动记录的指令和其他指令

(ADDI, x, y, t1)

LD R addr(x)

ADD R addr(y)

ST addr(t1) R

◆ 目标代码生成的过程

- 顺次扫描四元式，生成对应的目标指令序列

目标代码的种类

实际目标代码

机器代码

绝对地址机器代码：绝对机器语言代码即能够立即执行的机器语言代码，代码中所有地址均已定位，编译后可直接执行。

可重定位的机器代码：当程序执行时，必须由连接装入程序把它们和一些运行时子程序连接起来，转换成可立即执行的机器语言代码。

汇编代码必须将汇编语言代码转换成可执行的机器语言代码才可以执行。

虚拟目标代码：虚拟机上的目标程序。

在本地机器上具备虚拟机的解释器。

虚拟目标机的指令系统

目标机指令系统的指令分成如下的形式：

- ◆ 存/取指令： ST / LD
- ◆ 输入输出： IN / OUT
- ◆ 运算型指令： ADD SUB MUL DIV GT GE...
- ◆ 转移型指令： JMP JMPT JMPF
- ◆ 地址运算指令和块传递指令： LEA MOVEB

虚拟目标机的指令系统

指令名称	指令形式	指令名称	指令形式
读	IN R	除	DIV R, A
写	OUT R	条件真转移	JMPT R, A
取值	LD R, A	条件假转移	JMPF R, A
存值	ST A, R	无条件转移	JMP A
加	ADD R, A	取址	LEA R, A
减	SUB R, A	块传送	MOVEB A1, A2, S
乘	MULT R, A		

虚拟目标机的寄存器

- ◆ 寄存器是CPU内部的元件，寄存器拥有非常高的读写速度，它们可用来暂存指令、数据和地址。
- ◆ 寄存器种类：累加器、变址器、通用寄存器等等
- ◆ 在虚拟目标机中，取出几个寄存器作为地址计算专用的寄存器，分别为SP、TOP、SP0；其他寄存器用R1、R2...表示

四元式种类

- 运算型: (op, A, B , T)
- 赋值: (Assig, A, size, B)
- 输入/输出: (Read,-,-,A) (Write,-,-,t)
- 跳转和标号:
 - (JMP/JMPF/JMPT, -,-, L)、 (Label, -,-, L)、
 - (THEN,t,-,-)、 (ELSE,-,-,-)、 (ENDIF,-,-,-)、
 - (WHILE,-,-,-)、 (DO,t,-,-)、 (ENDWHILE,-,-,-)
- 函数相关
 - (ENTRY, Lf, size, level)
 - (ENDFUNC, _, _, _)
 - (ValAct/VarAct, X.arg, offset, size)
 - (CALL, Lf , true, result)
 - (RETURN, -, -, t/-)

四元式转化为目标指令

四元式等价地转换成目标指令需要用到两个栈：

- ◆ 标号定位栈L1：定位性标号是为了某些转移提供地址，需要把暂时没用到的标号存在栈中，例如while四元式可以对应一个嵌套的循环，在定位产生时还没产生跳转指令，它的地址还没用到，为了让后面能用到需要用栈把标号保存下来
- ◆ 目标指令地址栈L2：在有些产生跳转指令的时候，转移地址暂时无法确定，例如do四元式，不知道后面的转移地址，则把当前目标指令地址存到栈里，在知道转移地址以后，回填这个指令地址。回填地址是编译中的一项非常重要的技术

运算型四元式目标代码生成

运算型四元式 (ω , X, Y, T) : 对应生成3条指令

LD R X; 把X取到R中

ω R Y; 根据运算符对应的运算生成指令, 结果存入R

ST T R; 将R中的内容存入T中

两点重要的解释:

- ❖ 目标指令中X, Y, T对应的一个具体的地址, 如 $sp+off_x$
- ❖ 如果X、Y为间接变量, 这里的寻址方式为间接, 如X为间接变量, 用*X表示它是一个间接寻址方式。

LD R *off_x[sp];

ω R *(off_y+d[L])[sp];

ST *off_T[sp] R;

LD R *X;

ω R *Y;

ST *T R;

赋值型四元式目标代码生成

赋值型的四元式(ASSIG, x ,-, y):
生成的目标指令是两条

LD R X; 把 x 取到寄存器中

ST Y R; 把寄存器中的内容存到 y 中

注意的问题和前面是一样的(地址、间接寻址)。

输入输出四元式目标代码生成

- ◆ 输入语句四元式的一般形式为：

(READ, _, _, A)

表示将一个外部值读入到变量A中。

它生成的代码如下：

IN R;

ST A R;

输入输出四元式目标代码生成

- ◆ 输出语句的四元式形式一般为：

(WRITE, _, _, A)

表示将变量A的值输出。

它生成的目标代码为：

LD R A;

OUT R;

条件语句目标代码生成

- ◆ 条件语句四元式的翻译 (then else endif) :
(THEN, x, -, -) 起跳转作用,
先是产生一条半指令, 待处理到else四元式时, 将半条指令补全。

[n] LD R X; 一条

[n+1] JMPF R * (地址) ; 半条

同时把转移指令地址n+1压入L2栈

条件语句目标代码生成

(ELSE, -, -, -), else起到定位和转移的作用, 因此要产生跳转指令, 并完成 (THEN, t, -, -) 生成指令的地址回填。

[m-1].....

地址回填, 从L2栈中取出指令: {[n+1]JMPF R * (地址) }
将m+1作为地址加入该指令: [n+1]JMPF R *(m+1)

[m] JMP * (地址)

因为循环的结束endif还未处理, 所以将m压入L2栈,
待回填

[m+1].....

条件语句目标代码生成

(ENDIF, , ,), 起定位作用, 不产生目标指令,
只回填else产生的半条指令JMP的地址

从L2栈弹出 { [n] JMP * (地址) }

假设当前生成完 第m条指令, 即将生成第m+1指令
则将第n条指令的跳转地址部分补充完整 JMP * (m+1)

While循环语句目标代码的生成

- ◆ while循环语句的翻译：对应WHILE、DO、ENDWHILE三条四元式的翻译
- ◆ (WHILE, -, -, -) 起到一个定位作用，不产生指令，
[n]... ...
[n+1]记录当前目标指令地址n+1，压入L1栈

While循环语句目标代码的生成

- ◆ 循环语句的翻译while:
(DO, X, -, -) 生成一条半指令

[m] LD X R;

[m+1] JMPF R * (地址) ;

m+1压入L2栈, 后面遇到endwhile再回填

While循环语句目标代码的生成

- ◆ 循环语句的翻译while:
(ENDWHILE, -, -, -) 既是标记又是跳转

L1栈顶出栈, 生成一条指令

[k] JMP *L1(top);

将地址k+1回填给L2栈顶代码:

JMPF R * (k+1) L2栈顶出栈

[k+1].....

例子P183-5 试写出下述程序段的目标代码，其中变量均为非形参整型变量。

```
while (x < y)
{
    y = y + 1;
    if (y > 0)    y = y-x;
    else while (y < 0)
        y = y + x;
}
```

中间代码结构

```
(WHILE,-,-,-)
(LT , x, y, t1)
(DO,t1,-,-)
(ADDI,y,1,t2)
(ASSIG,t2,-,y)
(GT , y, 0, t3)
(THEN, t3,-,-)
(SUBI,y,x,t4)
(ASSIG,t4,-,y)
```

```
(ELSE,-,-,-)
(WHILE,-,-,-)
(LT , y, 0, t5)
(DO,t5,-,-)
(ADDI,y,x,t6)
(ASSIG,t6,-,y)
(ENDWHILE , - , - , - )
(ENDIF,-,-,-)
(ENDWHILE , - , - , - )
```

目标代码

1. **L1(top)=1**

LD R,x

2. LT R,y

3. ST t₁, R

4. LD R,t₁

5. JMPF R, * (34)

L2(top)=5

6. LD R,y

7. ADD R,1

8. ST t₂, R

9. LD R,t₂

10. ST y, R

11. LD R,y

12. GT R,0

13. ST t₃, R

14. LD R,t₃

15. JMPF R,* (22)

L2(top)=15

16. LD R,y

17. SUB R,x

18. ST t₄, R

19. LD R,t₄

20. ST y, R

21. JMP *(33)

回填22至L2(top)

L2(top)=21

22. **L1(top)=22** LD R,y

23. LT R,0

24. ST t₅, R

25. LD R,t₅

26. JMPF R, *(33)

L2(top)=26

27. LD R,y

28. ADD R,x

29. ST t₆, R

30. LD R,t₆

31. ST y, R

32. JMP 22:L1(top)

回填33至L2(top)

33. 回填33至L2(top)

JMP 1:L1(top)

回填34至L2(top)

```
while (x < y)
{
    y = y + 1;
    if (y > 0)    y = y-x;
    else while (y < 0)
        y = y + x;
}
```

标号和跳转四元式的目标代码生成

- ◆ 标号语句：(lable,-,- L)，将当前目标代码地址存入L对应的标号表单元。
- ◆ goto语句：(JMP,-,-,L) 产生一个无条件转移指令 $\text{JMP } * (L)$ ，对应的地址暂时为一个指向标号表L的指针。
- ◆ 目标代码全部生成以后，对所有由(JMP,-,-,L)生成的目标指令，依据标号表中对应的目标代码地址信息，进行地址回填。

函数相关四元式的代码生成

◆ 函数相关四元式

(VALACT , a, off, size)

(VARACT , a, off, size)

(CALL, Lf, true/false, result)

(ENTRY, Label, size, Level)

(RETURN,-,-,t/-)

(ENDFUNC, -,-,-)

- ◆ 翻译时，需要加进一些额外的代码用于完成如下工作：
 - 函数调用时：申请新AR空间，保存寄存器状态信息，保存返回地址、返回值、动态链指针，构建变量访问环境，传递参数，转向相应过程体等
 - 函数返回时：恢复寄存器状态信息，传递返回值，按照返回地址返回,释放AR等

函数声明相关四元式的翻译

- ◆ 过程函数声明部分的四元式处理:
- ◆ (ENTRY, Q, -, -)

Name	TypePtr	Kind	Level	off	Parm	Class	<u>Code</u>	Size	Forward
------	---------	------	-------	-----	------	-------	-------------	------	---------

入口四元式不产生任何指令，而是把当前指令地址填入函数信息表中。

◆ 过程函数声明部分的四元式处理:

(ENDFUNC, -, -, -) / (return, -, -, t)

1: 生成一组读取命令, 即恢复寄存器的现场信息

2: 作废当前的活动记录, 由两个指令完成,
把当前的sp存给top, 把动态链指针存给sp

ST top sp;

LD sp *0(top);

3: 产生一条返回指令, 根据返回地址生成一个跳转指令。

JMP *1(top)

如果是返回语句 (return, -, -, t), 则要增加一个操作, 将返回结果存储到活动记录2 (top) 中。

LD R t

ST *2(top) R //top指针为调整后的

top →

.....

寄存器状态

过程层数

返回值

返回地址

动态链指针

sp →

调用者AR

函数调用相关四元式的翻译

值参四元式(ValACT , t , Offset , size)

a. 若t为间接变量, 则生成的目标代码为:

```
LD  R  * t ;
```

```
ST  Offset(top)  R
```

b. 若t 为直接变量, 则生成的目标代码为:

```
LD  R  t ;
```

```
ST  Offset(top)  R ;
```

c. 若t 为数组, 则生成成组传送的目标代码:

```
MOVEB  t  Offset(top)  size ;
```

函数调用相关四元式的翻译

变参四元式(VarACT , t , Offset , size)

a. 若t为直接变量，则生成的目标代码为：

LEA R t;

ST offset(top) R;

b. 若t为间接变量，则生成的目标代码为：

LD R t ;

ST offset(top) R;

函数调用相关四元式的翻译

(CALL , f, true , Result)

1. 生成填写变量访问环境指令
2. 把机器状态(寄存器内容)保存到活动记录的寄存器状态区中,一般应生成一组存的指令
3. 要填写管理信息. 首先填写过程层数. 从过程f的语义信息中取其层数, 填入到3 (top) 中, 生成指令为

```
LD R sem[f].level;
```

```
ST 3(top) R;
```

取出过程活动记录大小, 填入off-size (top) 中, 生成指令为

```
LD R sem[f].size;
```

```
ST off-size(top) R;
```

被调用者AR

top

临时变量区

局部变量区

形参区

变量访问环境

活动记录大小

寄存器状态

过程层数

返回值

返回地址

动态链指针

sp

(CALL , f, true , Result)

4. 填写动态链指针

ST 0(top) sp;

5. 填写返回值

LEA R,result;

ST 2(top) R;

6. 填写返回地址

[A] LD R A+5;

[A+1] ST 1(top) R ;

7. 生成过程活动记录

[A+2] ST sp top;

[A+3] ST top top + sem[f].size;

8. 生成转向过程f入口的指令

[A+4] JMP sem[f].code ;

[A+5]

被调用者AR

top

临时变量区

局部变量区

形参区

变量访问环境

活动记录大小

寄存器状态

过程层数

返回值

返回地址

动态链指针

sp

多寄存器的分配

为了产生更高效的目标代码，合理利用寄存器资源，需要构造一个寄存器状态表：

名字	状态	变量名
R1	0/1	X

当需要用到一个寄存器时，可以用一个函数查找寄存器表，检查是否存在空闲寄存器，若存在，则将空闲的寄存器分配，然后按存取计算操作来操作，同时把相应的寄存器的状态和占用者记录在寄存器状态表

多寄存器的分配

- ◆ 当所有寄存器都被占用时，涉及到寄存器的剥夺问题。剥夺算法优劣会影响目标程序的执行效率。
- ◆ 对寄存器的剥夺是把寄存器中的内容存入内存中。
- ◆ 理论上的最优是最远使用点方法，实现困难
- ◆ 局部最优是基本块内最远使用点方法

考虑寄存器分配的(+,A,B,t)的代码生成

(+, A, B, T)	B不在寄存器 且不活跃	B不在寄存器 且活跃	B在寄存器R _B
A不在寄存器	LD R _B B Add R _B A	LD R _A A LD R _B B Add R _B , R _A	LD R _A A Add R _A R _B
A在寄存器R _A	Add R _A B	LD R _B B Add R _B , R _A	Add R _B , R _A

对目标程序的评价

- ◆ 衡量目标代码质量的标准：
 - 在保证语义相等的情况下，生成的目标指令的条数越少越好，目标指令的运行速度越快越好
- ◆ 对内存的访问次数总和决定了目标程序的执行效率。
- ◆ 指令的执行代价：访问内存一次的代价为1，例如间接寻址指令的代价较高
- ◆ 多利用寄存器中的资源可以达到减少访问内存的效果。

临时变量地址分配

◆ 临时变量的特点

- 数量多，寿命短；
- 不优化，只定义一次，使用一次，且定义点和使用点在同一个基本块内；
- 公共表达式节省，将出现多次引用。

◆ 临时变量的存储空间

- $x := a + b - c + d$ ，则产生的中间代码为：

$(+, a, b, t1) \quad (-, t1, c, t2) \quad (+, t2, d, t3)$
 $(:=, t3, x)$

3个临时变量可共享一个临时单元。

临时变量地址分配

- ◆ 可以在中间代码生成或者目标代码生成时做；
- ◆ 静态分配法（共享法）

将临时变量的空间安排在AR栈区。计算出临时变量的活动区间，如果两个活动区间不相交，则可以共享同一单元。

- ◆ 动态分配法（随用随分配法）

过程调用时分配的AR区中不含临时变量部分，以后每当要保存临时变量的值时，动态分配到栈区。当临时变量所占的空间比较大时，这种方法比较好。

总结

- ◆ 目标程序的生成要考虑的问题：
 - ◆ 目标机的具体形式
 - ◆ 如何把四元式翻译成目标程序
 - ◆ 优化，如何高效的生成目标代码。
-
- ◆ 要注意的问题：
- 符号表一直存在到目标代码生成，目标代码生成之后没有符号表，运行的只是目标程序。